RALLYSTATS · DOCUMENTO INTERNO

RallyStats para Windows: suscripción y versión comercial

Qué hace falta para vender la versión de escritorio después del punto 3: cómo saber si una cuenta es premium en Windows, cómo cobrar desde Windows si RevenueCat no tiene SDK ahí, cómo distribuirla y en qué orden hacerlo, con los prompts para Claude Code.

Basado en el código de `volley_stats_app` revisado el 21 y 22 de octubre de 2026 (antes de implementar los puntos 3A y 3B) y en la documentación pública de RevenueCat, Paddle, Mercado Pago, Microsoft, Google Play y FlutterFire consultada el 23/10/2026.
Los montos y comisiones son de esas páginas a esa fecha y pueden cambiar. Lo impositivo es orientativo: hay que confirmarlo con un contador.

> **Hallazgo que cambia el plan:** el README de FlutterFire marca `firebase_core`, `firebase_auth` y `cloud_firestore` en Windows como "for development only. Production on Windows is not supported", y `cloud_functions` como no disponible. La versión comercial necesita una base de Firebase soportada en Windows antes que cualquier cobro (sección 3).

1. Resumen

Sí se puede vender la versión de Windows, pero no alcanza con cambiar la suscripción. Hay que resolver cuatro cosas en este orden: una base de Firebase soportada en Windows, leer el estado premium real, cobrar fuera de las tiendas y distribuirla firmada. El punto 3 le da el sentido comercial: con el sync, los partidos y los equipos del celular aparecen en la PC.

- **RevenueCat sigue siendo la fuente de verdad.** No tiene SDK para Windows, pero su API REST (desde una Cloud Function) da el mismo estado que ve Android. Quien paga en Play queda premium en la PC sin pagar dos veces.
- **Cobro recomendado: Paddle**, con la integración oficial de RevenueCat. Paddle es merchant of record (cobra, factura y liquida impuestos en cada país) y Argentina no está entre sus países excluidos. Alternativa si Paddle no aprueba la cuenta: **Mercado Pago** con un webhook propio. Stripe queda descartado salvo con una empresa en EE.UU.
- **Distribución recomendada: Microsoft Store.** La cuenta de desarrollador individual es gratis, la Store firma el MSIX sin costo y, para apps que no son juegos, permite cobrar con un sistema propio quedándose con el 100%.
- **Dos arreglos chicos que no se pueden saltear:** un deviceId estable en Windows (hoy se pierde al reinstalar y deja "dispositivos fantasma") y no mencionar ni linkear la compra de Windows desde la app de Android (política de pagos de Play).

Etapa	Qué resuelve	Depende de	¿Se publica sola?
W1	Firebase por REST en Windows (login, lugar de dispositivo, funciones)	Nada (mejor después del 3A)	Sí; mejora lo que ya existe
W2	Estado premium real y deviceId estable	3A y W1	Sí; los que pagan en Play ya pueden usar Windows
W3	Cobro con Paddle (W3b: Mercado Pago)	W2 + cuenta de Paddle aprobada	Sí
W4	Distribución por Microsoft Store	W2 (en paralelo con W3)	Sí
W5	Sync de partidos y equipos en Windows	3B y W1	Sí; opcional para el lanzamiento

2. Cómo está hoy la versión de Windows

Hoy la app compila y anda en Windows, pero varias piezas se resolvieron pensando en que no se vendía. Cada una de estas es un bloqueo para cobrar.

Pieza	Qué hace hoy en Windows	Problema para venderla
Suscripción	`isRevenueCatSupported` es false en Windows y `SubscriptionController.refresh()` fuerza `isPremium = true`	Todo es gratis en Windows
Límite de dispositivos	`_computeDeviceLimit` cae a 1 y `_computeConfirmedDeviceLimit` devuelve null por ser Windows	Una cuenta con complementos no puede sumar la PC a su límite real
deviceId	Id al azar guardado en Hive (no hay equivalente de `ANDROID_ID`)	Reinstalar crea un "dispositivo fantasma" que ocupa un lugar; con el plan de 1 dispositivo deja al usuario afuera
Login y Firestore	`firebase_auth` y `cloud_firestore` por FlutterFire	FlutterFire los marca en Windows como solo para desarrollo; producción no soportada (sección 3)
Cloud Functions	`redeem_code_service.dart` usa `cloud_functions`	No disponible en Windows: "Tengo un código" no puede funcionar ahí
Sync (3B)	El prompt 3B no activa el sync en Windows: queda para W5	Sin sync de partidos y equipos, la versión de PC pierde su principal atractivo
Distribución	.zip e instalador de Inno Setup sin firma	SmartScreen bloquea o asusta; no hay actualizaciones automáticas
Builds ya repartidas	El premium forzado está en el código local	Quien tenga un instalador viejo sigue con premium gratis (sección 12)

3. El bloqueo principal: Firebase en Windows

La versión comercial no debería depender de FlutterFire en Windows. En la tabla de plataformas del README de FlutterFire, Core, Authentication, Cloud Firestore, Storage y Remote Config figuran en Windows con un asterisco cuya nota dice "(*) for development only. Production on Windows is not supported". Cloud Functions figura como N/A. Que hoy funcione en tu PC no cambia eso: un error en una versión futura no tiene a quién reclamarse, y el login y el control de dispositivos son justo lo que protege el cobro.

Opción	Cómo es	Evaluación
Seguir con FlutterFire en Windows	No tocar nada; ya funciona en desarrollo	**No recomendada** para cobrar: sin soporte en producción, sin Functions y con la cola offline sin garantías
Capa propia sobre las APIs REST de Firebase	Interfaz común; en Android sigue FlutterFire y en Windows se usan Identity Toolkit, Firestore REST y el protocolo HTTPS de las callables	**Recomendada**: son APIs públicas y soportadas, las reglas de seguridad aplican igual y la app usa pocas operaciones
Paquete de la comunidad (por ejemplo, firedart)	Cliente Dart de Auth y Firestore sin plugins nativos	**Posible**, pero suma una dependencia sin respaldo oficial en el camino del cobro

Qué usa la app y cómo se reemplaza en Windows

Operación de hoy	En Windows, por REST	Nota
Login, registro, recuperar contraseña	Identity Toolkit: `accounts:signInWithPassword`, `accounts:signUp`, `accounts:sendOobCode`	Los errores se traducen a los mensajes actuales
Sesión persistida	Refresh token + `securetoken.googleapis.com`	Guardado cifrado (DPAPI o Credential Manager), nunca en Hive en claro
Reautenticar y borrar la cuenta	`signInWithPassword` otra vez y `accounts:delete`	Mismo orden que hoy: primero los documentos, después el usuario
Reclamar el lugar de dispositivo	Firestore REST: `beginTransaction` + `commit`	Mismas reglas de seguridad, con el ID token
`_watchDeviceSlot` (escucha en vivo)	Sondeo del documento cada 30 s en primer plano y al volver	Por REST no hay escucha; el cierre remoto tarda hasta 30 s
`redeemPromoCode` y funciones nuevas	POST a `https://southamerica-east1-<proyecto>.cloudfunctions.net/<nombre>` con `{"data":…}` y el ID token	Es el protocolo HTTPS que usan las callables; no hace falta tocar el backend
Sync de partidos y equipos (3B)	`runQuery` + `commit` y una cola de salida propia en Hive	Etapa W5; por REST no hay cola offline del SDK

Efecto en los usuarios: al actualizar, los usuarios de Windows inician sesión una vez más (la sesión de FlutterFire no se migra). Con pocos usuarios de escritorio hoy, es un costo chico.

4. Arquitectura propuesta

Una sola fuente de verdad para "esta cuenta es premium": el entitlement `rallystats_pro` en RevenueCat, con el uid de Firebase como app user id. Cambia solo cómo llega cada plataforma a leerlo y dónde se paga.

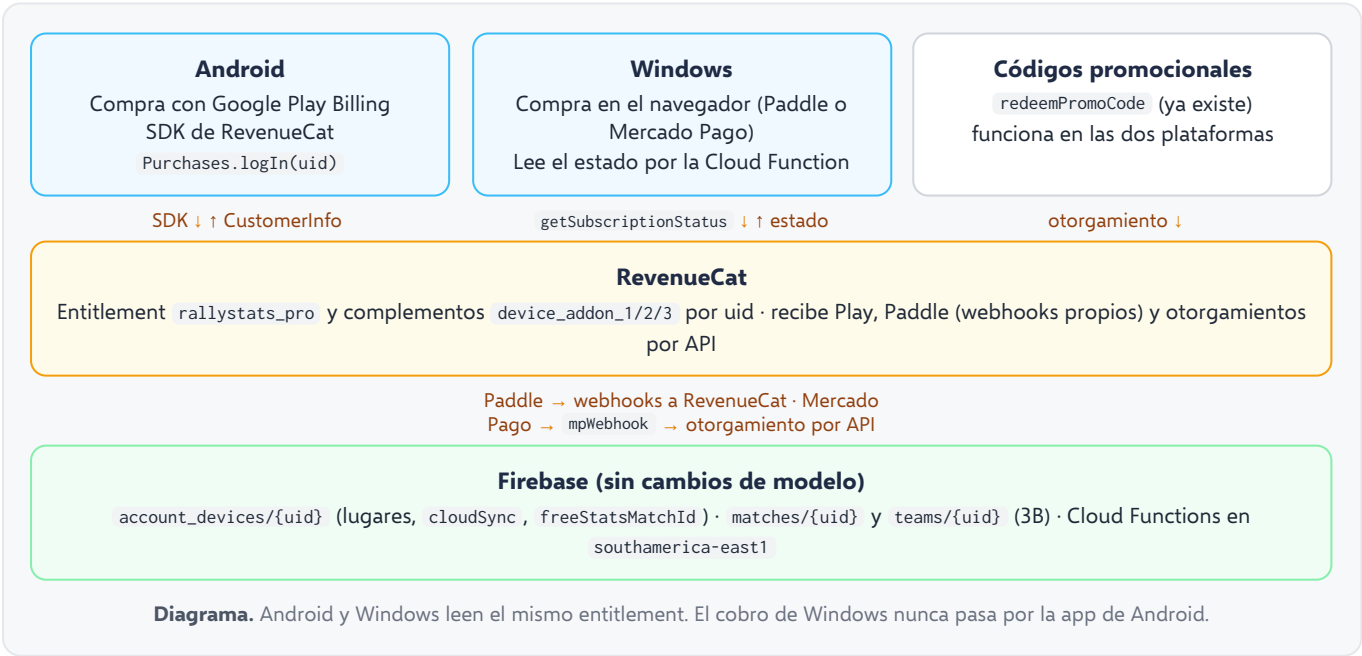


Diagrama. Android y Windows leen el mismo entitlement. El cobro de Windows nunca pasa por la app de Android.

La Cloud Function nueva, `getSubscriptionStatus`, evita meter la clave de RevenueCat en el cliente. Además reutiliza el código del backend que ya habla con su API (`redeemPromoCode`, `revenuecat.ts`). Devuelve las listas de entitlements y productos activos, y el cliente sigue calculando el límite con `deviceLimitFrom`. Así, la regla de cada posición de complemento vale igual en las dos plataformas.

5. Leer el estado premium en Windows

Con la etapa W2, una cuenta que paga en Play ya es premium en Windows. Sirve aunque todavía no haya cobro en Windows.

1. `platform_support.dart` pasa de un getter a dos: `canUseRevenueCatSdk` (Android/iOS) y `canReadSubscriptionStatus` (todas las plataformas, por SDK o por la función).
2. `SubscriptionController.refresh()` en Windows llama a `getSubscriptionStatus`. `isPremium` sale de `rallystats_pro` activo y `deviceLimit` de `deviceLimitFrom`. Se elimina el `isPremium = true` forzado.
3. Sin red se usa la caché por cuenta que agrega el 3A, igual que en Android.
4. `auth_service.dart`: el límite de dispositivos en Windows sale de la misma fuente. `_computeConfirmedDeviceLimit` sigue devolviendo null ante cualquier duda, para no autoexpulsar con un dato dudoso.
5. Hasta que exista el cobro (W3), una cuenta free en Windows ve las restricciones y un aviso de que la suscripción se contrata desde Android. Decirlo dentro de la app de Windows está permitido: la restricción de Play aplica solo a la app de Android.

> **Efecto inmediato:** la PC deja de ser premium para todos. Antes de publicar W2, avisá a quienes usan la versión de Windows sin pagar (testers, clubes de prueba) y, si corresponde, dales un código promocional.

6. Opciones para cobrar en Windows

Paddle es la opción con menos trabajo y menos riesgo fiscal; Mercado Pago es la que mejor le queda al usuario argentino. RevenueCat no tiene SDK para Windows, pero sí tiene integraciones web: su propio Web Billing (sobre Stripe), Stripe Billing y Paddle Billing, con links de compra hospedados.

Opción	Quién factura e impuestos	Integración con RevenueCat	Desde Argentina	Comisión publicada	Trabajo propio
Paddle	Paddle, como merchant of record	Oficial (webhooks automáticos, links de compra)	**Sí**: Argentina no está excluida; contempla vendedores individuales con su nombre legal	5% + USD 0,50 por transacción	Bajo: abrir el link con el uid y refrescar
Stripe / Web Billing de RevenueCat	Vos (o Stripe Managed Payments donde exista)	Oficial	**No** para un negocio con base en Argentina; habitualmente hace falta una LLC en EE.UU.	Según Stripe	Bajo, pero con un costo legal alto
Mercado Pago	Vos: facturás a cada cliente	No hay: webhook propio que otorga el entitlement por API	**Sí**, en pesos	Entre 1,49% y 6,99% + IVA según el plazo de liberación	Medio: funciones, renovaciones, cancelación e idempotencia
Comercio de Microsoft Store	Microsoft	No hay	Sí	Revenue share de Microsoft	Alto: otra integración de compras; no aporta frente a Paddle

Cuánto queda por suscriptor (aproximado, antes de impuestos)

Precio	Google Play (15%)	Paddle (5% + USD 0,50)
Mensual USD 2,99 (precio base del estudio de mercado)	USD 2,54 (85%)	USD 2,34 (78%)
Anual USD 29,90 (ejemplo)	USD 25,42 (85%)	USD 27,91 (93%)

Con un precio mensual bajo pesa el cargo fijo de Paddle; con el plan anual pasa a ser más barato que Play. Conviene destacar el plan anual en Windows. Las ventas de Paddle suman a los ingresos que mide RevenueCat igual que las de Play. Mercado Pago no entra en la tabla porque cobra en pesos y su costo depende del plazo de liberación y de los impuestos provinciales.

> **Recomendación:** pedir la aprobación de Paddle primero (sección 16). Si la aprueban, W3; si no, W3b con Mercado Pago solo para Argentina. Las dos variantes leen el estado de la misma forma (W2), así que elegir una no condiciona la otra.

7. Flujo de compra con Paddle

La app de Windows nunca maneja datos de tarjeta: abre el link de compra de RevenueCat con el uid y, al volver, pregunta el estado.

1. El usuario free toca "Mensual" o "Anual" en el paywall de Windows.
2. La app abre en el navegador el Web Purchase Link de RevenueCat con el uid de Firebase como app user id. Sin uid no se abre nada.
3. El checkout de Paddle cobra, factura y aplica los impuestos del país del comprador.
4. Paddle avisa a RevenueCat por webhook y RevenueCat activa `rallystats_pro` para ese uid.
5. Al volver a la app (o con "Ya pagué"), `refresh()` consulta `getSubscriptionStatus`. Si todavía no figura, se reintenta a los 5, 15 y 30 s.
6. Android ve el mismo entitlement en su próximo `refresh()`: la cuenta es premium en todos sus dispositivos.

Situación	Qué pasa
Renovación	La cobra Paddle; RevenueCat extiende el vencimiento por webhook. La app no hace nada.
Cancelar	Desde Windows, "Gestionar" abre el `managementUrl` que devuelve la función (portal de Paddle). También desde el mail de Paddle.
Reembolso o contracargo	Lo resuelve Paddle; RevenueCat revoca el entitlement y la app lo ve en el próximo refresh.
Ya es premium por Play	Windows no ofrece comprar y muestra que la suscripción viene de Google Play. Nadie paga dos veces.
Pruebas	Paddle necesita cuentas separadas de sandbox y producción, y RevenueCat ignora los webhooks simulados: las renovaciones se prueban con compras reales de sandbox.

Requisitos de Paddle que cuenta la doc de RevenueCat: Paddle Billing (no Classic), una API key con permisos de transacciones, tokens de cliente, notificaciones y sesiones del portal, y los dominios de pago aprobados. Una compra lleva un solo producto.

8. Alternativa: Mercado Pago

Mercado Pago sirve para cobrar en pesos, pero todo lo que en Paddle hace RevenueCat lo tenés que hacer vos. Sus suscripciones (preapproval) cobran automáticamente con tarjeta de crédito, débito o dinero en cuenta; el cliente no necesita cuenta de Mercado Pago y los pagos rechazados se reintentan.

1. `createMpSubscription` (callable) crea la suscripción para el uid (`external_reference` = uid) y devuelve el link de pago.
2. La app de Windows lo abre en el navegador.

3. `mpWebhook` recibe cada notificación, valida la firma y, por cada pago aprobado, otorga `rallystats_pro` por el período del plan + 3 días de gracia con la API de RevenueCat (el mismo mecanismo que los códigos promocionales).
4. Cada pago se registra en `mp_payments/{paymentId}` para no otorgarlo dos veces.
5. Si un pago se rechaza o se cancela la suscripción, no se extiende nada y el entitlement vence solo.

A favor	En contra
Cobro en pesos, que en Google Play no se puede	Facturás vos a cada cliente y te ocupás de los impuestos
Medio de pago conocido por el entrenador argentino	Renovaciones, gracia, cancelación e idempotencia son código propio
Comisión baja si se acepta liberar a 35 días	Solo sirve para Argentina; para otros países seguiría haciendo falta Paddle

9. Complementos de dispositivo en Windows

Recomendación: en Windows vender solo el plan base al principio. Quien necesite más dispositivos los suma desde Android, y el límite se lee igual en Windows porque `deviceLimitFrom` cuenta los complementos de cualquier tienda.

- En Paddle una compra lleva un solo producto: cada complemento sería una suscripción aparte, con el mismo problema de orden al cancelar que ya documenta `subscription_management_text.dart` para Play.
- Mezclar un plan base de Paddle con complementos de Play funciona técnicamente (el límite suma posiciones de cualquier origen), pero la gestión queda repartida entre dos tiendas y es difícil de explicar.
- Si la demanda aparece, los complementos de Paddle se suman después sin cambiar el modelo: son productos nuevos asociados a los entitlements `device_addon_1/2/3`.

10. Reglas entre plataformas

La app de Android no puede mandar a pagar afuera, pero sí puede usar un acceso pagado afuera. Es lo que dice la política de pagos de Google Play.

Regla	Qué implica en RallyStats
Play no permite "llevar a los usuarios a un método de pago que no sea el sistema de facturación de Google Play", ni con links directos a páginas que lleven a otro medio de pago	Ningún botón, link ni texto en Android para comprar en Windows, en la web o con Mercado Pago
Los links administrativos (cuenta, privacidad, ayuda) se permiten si no terminan en un medio de pago alternativo	Si la suscripción activa es de Paddle, Android dice dónde se gestiona, sin link
Una app puede dar acceso a contenido comprado fuera de Play	Lo pagado en Windows desbloquea premium en Android sin problema
Los programas de facturación alternativa (EEE, EE.UU., India, Corea) no mencionan Argentina	Para Android sigue todo por Play

Dentro de la app de Windows y en la web sí se puede ofrecer la compra y mencionar que también está la versión de Android.

11. Distribución

Recomendación: publicar en Microsoft Store como canal principal y dejar el instalador como alternativa. Para un desarrollador individual en Argentina es lo más barato y lo que menos fricción le genera al usuario.

Tema	Microsoft Store (MSIX)	Instalador propio (Inno Setup)
Cuenta	Gratis para individuos desde septiembre de 2025, con verificación de identidad (documento + selfie)	No hace falta
Firma	Microsoft firma el MSIX gratis después de la certificación	Certificado OV de una CA: unos USD 150–300 por año, con la clave en un token o HSM. Azure Artifact Signing (USD 9,99 por mes) es solo para individuos de EE.UU. y Canadá
SmartScreen	Sin advertencias para lo instalado desde la Store	Advertencias hasta ganar reputación; desde 2024 un certificado EV ya no las evita
Actualizaciones	Automáticas	Manuales
Cobro	Apps que no son juegos pueden usar su propio sistema y quedarse con el 100%	Libre

A verificar en W4: dónde guarda Hive los datos en una instalación MSIX frente a una de Inno Setup (`getApplicationDocumentsDirectory`), para que pasarse a la Store no pierda partidos.

12. deviceId estable y builds ya repartidas

Windows necesita un id de dispositivo que sobreviva a reinstalar, igual que Android con `ANDROID_ID`. Hoy es un id al azar en Hive: desinstalar y reinstalar genera uno nuevo y el viejo queda ocupando un lugar en `account_devices`, que con el plan de 1 dispositivo deja al usuario afuera de su propia cuenta.

- **Fuente:** `WindowsDeviceInfo.deviceId` de `device_info_plus`, que es el "Device ID" de la Configuración de Windows (`HKLM\SOFTWARE\Microsoft\SQMClient\MachineId`). Como respaldo, `HKLM\SOFTWARE\Microsoft\Cryptography\MachineGuid`.
- **Migración:** el mismo esquema de `legacy_device_id` que ya existe para Android, así el lugar reclamado con el id viejo se reconoce y no se pierde contra el límite.
- **Si no se puede leer ninguno,** se mantiene el esquema actual.

> **Builds ya repartidas.** El premium forzado vive en el código local, así que quien tenga el .zip o el instalador actuales sigue con premium gratis aunque se publique W2. No hay forma limpia de cortarlo sin afectar también a Android. Como hasta ahora la versión de Windows no se comercializó y solo la tienen testers, la propuesta es aceptarlo (decisión del prompt W2). Si hiciera falta cortarlo, la palanca sería una regla de Firestore que exija un dato nuevo en el reclamo de lugar, junto con una versión mínima en Android.

13. Legales, impuestos y facturación

Con Paddle la parte fiscal se simplifica mucho; con Mercado Pago queda toda de tu lado. Todo lo de esta sección es orientativo: confirmalo con un contador antes de cobrar.

Tema	Con Paddle	Con Mercado Pago
A quién le vendés	A Paddle, que revende al cliente final como merchant of record	A cada cliente
Facturación	Una factura a Paddle por cada liquidación (exportación de servicios)	Una factura por cobro a cada cliente
Impuestos del comprador	Los calcula, cobra y liquida Paddle en cada país	Los resolvés vos
Moneda	Precios en USD (Paddle convierte a moneda local; para Argentina, ARS con impuestos incluidos)	ARS

Tema	Con Paddle	Con Mercado Pago
Términos y privacidad	Hay que sumar al procesador elegido en la política de privacidad y en los términos (`lib/legal`), con su política de reembolsos. Paddle además pide en el sitio: descripción del producto, precios, términos, reembolsos y privacidad, con el nombre legal del titular y HTTPS.	

14. Riesgos

Riesgo	Impacto	Cómo se mitiga
FlutterFire en Windows falla en producción	**Alto**	W1: capa propia sobre REST antes de cobrar
Paddle no aprueba la cuenta o el dominio	**Medio**	Pedir la aprobación antes de programar W3; plan B con Mercado Pago (W3b)
El webhook tarda y el usuario cree que pagó de más	**Medio**	Reintentos y un mensaje claro al volver; "Ya pagué" para reintentar a mano
Doble suscripción (Play + Paddle)	**Medio**	No se ofrece comprar si la cuenta ya es premium por cualquier tienda
Rechazo de una versión de Android por mencionar la compra externa	**Alto**	Revisión de textos en W3: nada de links ni precios externos en Android
Dispositivos fantasma por reinstalar en Windows	**Medio**	deviceId estable (W2)
Cierre remoto más lento en Windows (sondeo cada 30 s)	**Bajo**	Aceptado; también se revisa al volver a primer plano
Cambio de comisiones o políticas de terceros	**Medio**	RevenueCat como capa común: cambiar de procesador no toca el modelo de premium

15. Plan por etapas

Etapa	Entregable	Backend	Cómo se prueba
W1	Auth, lugar de dispositivo y funciones por REST en Windows	Sin cambios	Login, cierre remoto, borrar cuenta y "Tengo un código" en una PC limpia
W2	Estado premium real, límite de dispositivos y deviceId estable	`getSubscriptionStatus`	Cuenta de Play premium en la PC; cuenta free restringida; reinstalar sin fantasma
W3	Compra y gestión con Paddle	Sin cambios (webhooks en RevenueCat)	Compra, renovación, cancelación y reembolso en sandbox; verlo en Android
W3b	Compra con Mercado Pago (si Paddle no aprueba)	Tres funciones + `mp_payments`	Webhook repetido sin doble otorgamiento; vencimiento al cancelar
W4	MSIX y ficha en Microsoft Store	Sin cambios	Instalar desde la Store, actualizar y conservar los datos
W5	Sync de partidos y equipos en Windows	Sin cambios (reglas del 3B)	Partido terminado o equipo editado en el celular aparece en la PC y al revés, con y sin red

Orden sugerido: 3A → 3B → W1 → W2 → (W3 o W3b) + W4 → W5. W1 y W2 se pueden empezar mientras se valida el 3B, porque no tocan el sync.

16. Lo que tenés que hacer vos (fuera del código)

- **Paddle:** crear la cuenta (Paddle Billing), pedir la verificación con tu nombre legal y aprobar el dominio de pago. Para eso el sitio tiene que tener descripción, precios, términos, reembolsos y privacidad (sirve la landing page de RallyStats).
- **Paddle:** crear los productos (mensual y anual) en sandbox y en producción; generar la API key con los permisos que pide RevenueCat y desactivar los mails de carrito abandonado.
- **RevenueCat:** conectar Paddle, crear los productos y asociarlos a `rallystats_pro`, y crear el Web Purchase Link.
- **Microsoft Partner Center:** registrarte como individuo (gratis), reservar el nombre de la app y pasarle a Claude Code la identidad del paquete (W4).
- **Contador:** confirmar cómo facturar las liquidaciones de Paddle (o los cobros de Mercado Pago) según tu situación fiscal.
- **Backend:** desplegar las funciones y reglas nuevas cuando estén aprobadas (Claude Code no despliega).
- **Comunicación:** avisar a los testers de Windows que la versión nueva deja de ser premium gratis y, si querés, darles códigos.

17. Cómo probarlo

ID	Caso	Resultado esperado
WP1	Login, cierre de sesión y recuperar contraseña en una PC sin entorno de desarrollo	Igual que en Android; la sesión sobrevive a cerrar la app
WP2	Borrar la cuenta desde el celular con la PC abierta	La PC cierra sesión en menos de 30 s o al volver a primer plano
WP3	Límite lleno: plan de 1 dispositivo, sesión en el celular, login en la PC	La PC rechaza el login con el mensaje de siempre
WP4	Cuenta premium de Play abre la PC	Premium sin comprar nada; el paywall no ofrece comprar
WP5	Cuenta free en la PC	Ve las restricciones; sin red usa la caché de la cuenta
WP6	Desinstalar y reinstalar en la PC	No queda un dispositivo fantasma; el lugar se reconoce
WP7	"Tengo un código" en la PC	Otorga premium o el complemento, como en Android
WP8	Compra mensual en el sandbox de Paddle	Premium en la PC tras volver (con reintentos) y en Android en el próximo refresh
WP9	Cancelar desde "Gestionar" y esperar el vencimiento	Sigue premium hasta la fecha y después pasa a free en las dos plataformas
WP10	Reembolso en Paddle	Premium revocado en el próximo refresh
WP11	Android con una suscripción de Paddle	Muestra dónde se gestiona, sin links ni precios
WP12	Revisar todos los textos de Android	Ninguna mención ni link de compra fuera de Play
WP13	Instalar desde la Store sobre una instalación de Inno Setup	Los partidos, equipos, jugadas y fixture siguen ahí
WP14	Mercado Pago (si aplica): pago aprobado, webhook repetido, pago rechazado y cancelación	Se otorga una sola vez, el rechazo no extiende y la cancelación deja vencer

18. Prompts para Claude Code

Cada prompt se pega tal cual en Claude Code, en el orden de la sección 15. Las líneas "Decisión:" fijan una decisión de producto: si se decide otra cosa, cambialas antes de usarlo. Los mismos prompts están en `RallyStats-Prompts-Funcionalidades.txt`, sección W.

W1 · BASE DE FIREBASE SOPORTADA EN WINDOWS

```
Repo: C:\Users\Usuario\volley_stats_app. Leé CLAUDE.md y documents/RallyStats-Windows-Comercial.pdf (secciones 3 y 4).
Antes de escribir código, presentame un plan breve para aprobar.
Contexto: en el README de FlutterFire, firebase_core, firebase_auth y cloud_firestore figuran en Windows como "(*) for development only. Production on Windows is not supported", y cloud_functions como N/A. Para vender la versión de Windows, la app no puede depender de eso.
Decisión: en Android no cambia nada (siguen FlutterFire y el SDK de RevenueCat). En Windows, Auth, Firestore y las Cloud Functions se usan por sus APIs REST públicas, detrás de una interfaz común.
1. Inventario: listá cada uso de FirebaseAuth, FirebaseFirestore y FirebaseFunctions en lib/ (auth_service.dart, redeem_code_service.dart y lo que hayan sumado 3A y 3B) y mostrámelo antes de seguir.
2. Interfaces, elegidas en un solo lugar según la plataforma (lib/utils/platform_support.dart):
   - AuthGateway: signIn, register, signOut, sendPasswordResetEmail, reauthenticate, deleteUser, currentUser (uid, email), authStateChanges, getIdToken.
   - AccountDevicesStore: get, la transacción de reclamo de lugar, actualizar un campo, borrar el documento y watch.
   - FunctionsGateway: call(nombre, data).
   La implementación FlutterFire es la de hoy, sin cambios de comportamiento. La implementación REST es solo para Windows.
3. REST en Windows:
   - Auth: Identity Toolkit (accounts:signInWithPassword, accounts:signUp, accounts:sendOobCode, accounts:delete) y renovación del token con securetoken.googleapis.com. Reautenticar = volver a llamar a signInWithPassword. El refresh token se guarda cifrado (verificá que flutter_secure_storage en Windows use DPAPI o el Credential Manager), nunca en Hive en claro. authStateChanges es un Stream propio. Los errores (EMAIL_NOT_FOUND, INVALID_PASSWORD, EMAIL_EXISTS, etc.) se traducen a los mismos mensajes que hoy.
   - Firestore: API REST v1 (documents get, patch, delete, y beginTransaction + commit para el reclamo de lugar y para freeStatsMatchId del 3A), autenticada con el ID token: las reglas de seguridad aplican igual. watch = sondeo cada 30 s con la app en primer plano y al volver a primer plano, cortado al cerrar sesión, con el mismo comportamiento de cierre que hoy tiene _watchDeviceSlot.
   - Functions: protocolo HTTPS de las callables: POST a https://southamerica-east1-<proyecto>.cloudfunctions.net/<nombre> con {"data": ...} y Authorization: Bearer <ID token>; respuesta {"result": ...} o {"error": ...}. Con esto "Tengo un código" (redeemPromoCode) pasa a funcionar en Windows.
   - apiKey y projectId salen de firebase_options.dart para Windows (verificá que exista esa entrada; si no, decime de dónde tomarlos).
   - Sin red: los mismos mensajes que hoy (patrón del login).
4. En Windows no se inicializa ni se llama a firebase_auth, cloud_firestore ni cloud_functions. Verificá si el build de Windows igual necesita los plugins registrados; si se pueden excluir de Windows sin romper Android, excluilos.
5. Decisión: al actualizar, los usuarios de Windows inician sesión una vez más (la sesión de FlutterFire no se migra). El lugar de dispositivo se conserva porque el deviceId no cambia en este prompt (eso es W2).
Tests con un cliente HTTP falso: login correcto y con cada error; refresh de un token vencido; transacción de reclamo con lugar libre, con el mismo dispositivo y con el límite lleno (DeviceConflictException); sondeo que detecta el documento borrado o el dispositivo quitado y cierra la sesión; callable con resultado y con error.
Terminar con flutter analyze y flutter test en verde y flutter build windows --release. Dame los pasos para probar en la PC: login, cierre de sesión, cierre remoto, borrar cuenta y "Tengo un código". No hagas commit.
```

W2 · ESTADO PREMIUM REAL Y DEVICEID ESTABLE EN WINDOWS

```
Repos: C:\Users\Usuario\volley_stats_app y C:\Users\Usuario\volley_stats_app_backend. Leé el CLAUDE.md de ambos y documents/RallyStats-Windows-Comercial.pdf (secciones 4, 5 y 12). Requiere el 3A (caché de la suscripción por cuenta) y el W1: si falta alguno, pará y avisame.
Antes de escribir código, presentame un plan breve para aprobar.
Backend:
1. Nueva Cloud Function onCall getSubscriptionStatus (southamerica-east1, mismo proyecto): verifica la sesión, llama a GET https://api.revenuecat.com/v1/subscribers/{uid} con la clave secreta que ya usa redeemPromoCode y devuelve {activeEntitlementIds, activeSubscriptionProductIds, premiumStore, premiumExpiresAt, managementUrl}. "Activo" = expires_date nula o futura. Sin caché propio. NO despliegues.
Frontend:
2. platform_support.dart: reemplazá isRevenueCatSupported por dos getters: canUseRevenueCatSdk (Android/iOS) y canReadSubscriptionStatus (todas las plataformas: por SDK o por la función). Revisá cada uso actual y elegí el que corresponde.
```

3. SubscriptionController en Windows: refresh() llama a getSubscriptionStatus por FunctionsGateway (W1). isPremium = rallystats_pro activo; deviceLimit = deviceLimitFrom(...) con las listas que devuelve la función (no dupliques esa lógica en TypeScript). Se elimina el isPremium = true forzado. Sin red, la caché por cuenta del 3A, igual que en Android.
4. auth_service.dart: _computeDeviceLimit y _computeConfirmedDeviceLimit usan la misma fuente en Windows y dejan de caer a 1 o a null por ser Windows. Se mantiene la regla: _computeConfirmedDeviceLimit devuelve null ante cualquier duda (sin red, error de la función).
5. Windows, hasta que exista W3: si la cuenta no es premium, el paywall dice que la suscripción se contrata desde la app de Android y no ofrece comprar. "Restaurar compras" se reemplaza por "Actualizar estado". Si la cuenta es premium, SubscriptionScreen muestra de qué tienda viene (premiumStore).
6. deviceId estable en Windows (storage_service.dart, _resolveDeviceId): usar el Device ID del sistema (device_info_plus, WindowsDeviceInfo.deviceId = HKLM\SOFTWARE\Microsoft\SQMClient\MachineId) y, si no está, HKLM\SOFTWARE\Microsoft\Cryptography\MachineGuid. Mismo esquema de legacy_device_id que ya existe para ANDROID_ID, para migrar el lugar reclamado con el id al azar viejo sin perderlo contra el límite. Si no se puede leer ninguno, el esquema de hoy.
7. Decisión: las builds de Windows ya repartidas (con premium forzado) se aceptan como están; no se bloquean.
Tests: deviceLimitFrom con las listas del backend; refresh sin red usa la caché; una cuenta free en Windows ve las restricciones; una cuenta premium de Play es premium en Windows; resolución del deviceId en Windows y migración del legacy.
Actualizá CLAUDE.md (Suscripción premium, Login y control de dispositivos) y el manual según el proceso de release.
Terminar con flutter analyze y flutter test en verde (y npm run build en functions). No despliegues. No hagas commit.

W3 · COBRO EN WINDOWS CON PADDLE (VÍA REVENUECAT)

Repo: C:\Users\Usuario\volley_stats_app. Leé CLAUDE.md y documents/RallyStats-Windows-Comercial.pdf (secciones 6, 7, 9 y 10). Requiere W2. Antes de empezar confirmame que ya están hechos los pasos manuales de la sección 16 (cuenta de Paddle aprobada, productos en Paddle y en RevenueCat, integración Paddle en RevenueCat y el Web Purchase Link); si no, pará.
Antes de escribir código, presentame un plan breve para aprobar.
Decisión: Paddle es el merchant of record. En Windows se vende solo el plan base, mensual y anual; los complementos de dispositivo siguen siendo solo de Play por ahora.
1. Paywall y SubscriptionScreen en Windows: botones "Mensual" y "Anual" que abren en el navegador (url_launcher) el Web Purchase Link de RevenueCat con el uid de Firebase como app user id, en el formato que indica su documentación. Nunca se abre un link sin uid.
2. Al volver a la app (didChangeAppLifecycleState resumed) y con un botón "Ya pagué", refresh(). Si la compra todavía no figura, reintentar a los 5, 15 y 30 s (el webhook de Paddle a RevenueCat puede tardar) con un mensaje claro, y después dejar el botón para reintentar a mano.
3. Si la cuenta ya es premium por cualquier tienda, no se ofrece comprar: se muestra de dónde viene. Así nadie paga dos veces.
4. Gestionar en Windows: si premiumStore es Paddle, abrir managementUrl (de getSubscriptionStatus). Si es Play, decir que se gestiona desde Google Play en el celular.
5. Android: si la suscripción activa es de Paddle (la tienda del entitlement en CustomerInfo), SubscriptionScreen muestra "Tu suscripción se gestiona desde RallyStats para Windows o desde el mail de Paddle", sin links ni precios. En Android no se agrega ningún link ni mención para comprar fuera de Google Play (política de pagos de Play).
6. subscription_tiers.dart: sumá los ids de los productos de Paddle tal como los configures en RevenueCat, sin tocar los de Play. El entitlement sigue siendo rallystats_pro.
7. Legales: redactá los cambios de lib/legal (términos y política de privacidad) para sumar a Paddle como vendedor y procesador de pagos en Windows, con su política de reembolsos. Mostrámelos para aprobar ANTES de aplicarlos y usá el mecanismo de versión de la política que exista.
Tests: el link lleva el uid y sin uid no se abre; una cuenta premium no ve la compra; los reintentos después de volver; Android con un entitlement de Paddle no muestra links.
Actualizá CLAUDE.md, el manual y el README según el proceso de release.
Terminar con flutter analyze y flutter test en verde. Dame los pasos para probar una compra en el sandbox de Paddle de punta a punta (compra, renovación, cancelación y reembolso) y verla en Android. No hagas commit.

W3b · ALTERNATIVA: COBRO EN WINDOWS CON MERCADO PAGO

Usar solo si Paddle no aprueba la cuenta. Repos: C:\Users\Usuario\volley_stats_app y C:\Users\Usuario\volley_stats_app_backend. Leé el CLAUDE.md de ambos y documents/RallyStats-Windows-Comercial.pdf (sección 8). Requiere W2.
Antes de escribir código, presentame un plan breve para aprobar.
Decisión: precio en pesos, solo para Argentina. Mercado Pago no tiene integración con RevenueCat: cada pago aprobado otorga el entitlement por la API de RevenueCat, con el mismo mecanismo que redeemPromoCode.
Backend (secretos con defineSecret; NO despliegues):
1. createMpSubscription (onCall): crea una suscripción (preapproval) para el uid con el plan elegido, external_reference = uid y back_url, y devuelve el init_point.
2. mpWebhook (onRequest): valida la firma x-signature con el secreto de Mercado Pago; ante un pago aprobado de una suscripción, busca el uid por external_reference y otorga rallystats_pro por el período del plan + 3 días de gracia. Idempotente: cada payment id se guarda en mp_payments/{paymentId} y no se aplica dos veces. Un pago rechazado o una suscripción cancelada no extienden nada; el entitlement vence solo.
3. cancelMpSubscription (onCall): cancela el preapproval del uid.
4. Estado en mp_subscriptions/{uid} (preapproval id, plan, estado, próximo cobro). Reglas: solo lectura del dueño; escritura

```
solo desde las funciones.
Frontend Windows:
5. Paywall: botón que llama a createMpSubscription y abre el init_point en el navegador; al volver, refresh() con los mismos reintentos que W3.
6. Gestionar: muestra el estado de mp_subscriptions y un botón Cancelar con confirmación.
7. Legales: igual que W3 pero con Mercado Pago; mostrámelos antes de aplicarlos.
Tests con el emulador de functions: webhook con firma inválida rechazado, pago repetido sin doble otorgamiento, duración del otorgamiento, cancelación.
Terminar con flutter analyze, flutter test y npm run build en verde. No despliegues. No hagas commit.
```

W4 · DISTRIBUCIÓN EN MICROSOFT STORE (MSIX)

```
Repo: C:\Users\Usuario\volley_stats_app. Leé CLAUDE.md y documents/RallyStats-Windows-Comercial.pdf (sección 11). Puede hacerse en paralelo con W3.
1. Paquete msix (pub.dev) configurado en pubspec.yaml con la identidad de Partner Center (identity_name, publisher, publisher_display_name: te los paso yo) y la capability internetClient. Versión tomada de pubspec.
2. Datos: verificá dónde quedan las cajas de Hive en una instalación MSIX y en una de Inno Setup (getApplicationDocumentsDirectory) y que pasar de una a la otra no pierda datos. Si la ruta cambia, proponé una migración con el mismo cuidado que el 3A (copia previa, nada se borra) y mostrámela antes de implementarla.
3. Nada de autoactualización propia: la Store actualiza.
4. Proceso de release del CLAUDE.md: sumá el paso "dart run msix:create" (con store: true) y la subida a Partner Center. Inno Setup y el .zip quedan como alternativa.
5. Redactá el texto de la ficha de la Store (descripción corta, larga, novedades) en español, sin mencionar precios de Google Play.
Terminar con flutter analyze en verde y el .msix generado. No publiques nada. No hagas commit.
```

W5 · SYNC DE PARTIDOS Y EQUIPOS EN WINDOWS (DESPUÉS DEL 3B)

```
Repo: C:\Users\Usuario\volley_stats_app. Leé CLAUDE.md y documents/RallyStats-Windows-Comercial.pdf (secciones 3 y 15). Requiere 3B y W1.
Antes de escribir código, presentame un plan breve para aprobar.
Contexto: por REST no hay cola offline ni escuchas en vivo, así que en Windows el MatchSyncService del 3B necesita una implementación propia detrás de la misma interfaz.
1. Cola de salida en Hive por cuenta (outbox_box_<sufijo>): cada push pendiente (colección, id del partido o del equipo y updatedAt) se guarda antes de intentar subirlo y se borra solo cuando Firestore confirma. Se procesa al abrir, al volver a primer plano y cada 60 s mientras haya pendientes.
2. Pull por runQuery: serverUpdatedAt > cursor, para partidos y equipos, con las mismas reglas del 3B (gana el updatedAt más nuevo, deletedAt manda el partido a la papelera y oculta el equipo, nunca se borra lo que falta, no se toca un partido abierto, las fotos no viajan y se conserva el photoPath local).
3. Subida inicial reanudable con el mismo progreso que en Android.
4. El switch "Partidos y equipos en la nube" aparece en Windows.
Tests con un Firestore REST falso: push sin red queda en la cola y se envía al volver; la cola sobrevive a cerrar la app; pull incremental; mismas reglas de conflicto y papelera que el 3B; cambio de cuenta corta la cola sin escribir en la otra.
Terminar con flutter analyze y flutter test en verde. No hagas commit.
```

Fuentes

- FlutterFire, tabla de plataformas del README (consultado el 23/10/2026): github.com/firebase/flutterfire
- RevenueCat, Web overview: revenuecat.com/docs/web/overview
- RevenueCat, integración con Paddle: revenuecat.com/docs/web/integrations/paddle
- RevenueCat, API v1 Customers y Entitlements: revenuecat.com/docs/api-v1/customers · /entitlements
- Paddle, precios: paddle.com/pricing
- Paddle, países soportados: paddle.com/help/start/intro-to-paddle/which-countries-are-supported-by-paddle
- Paddle, aprobación de dominio: paddle.com/help/start/account-verification/what-is-domain-approval
- Mercado Pago, suscripciones: mercadopago.com.ar/herramientas-para-vender/suscripciones
- Google Play, política de pagos: support.google.com/googleplay/android-developer/answer/10281818
- Microsoft, registro gratis para individuos: blogs.windows.com/windowsdeveloper/2025/09/10/free-developer-registration-for-individual-developers-on-microsoft-store

- Microsoft, opciones de firma de código: learn.microsoft.com/windows/apps/package-and-deploy/code-signing-options
- device_info_plus, WindowsDeviceInfo.deviceId: pub.dev/documentation/device_info_plus